

AI Agents — Knowledge Base

A practical reference document for testing InsightAgent's PDF retrieval and RAG capabilities.

1. What Is an AI Agent?

An AI agent is a software system that uses a large language model (LLM) to reason about a goal, decide what actions are needed, use tools when necessary, and produce a result. Unlike a simple question-answering chatbot, an agent can follow a multi-step process and interact with external systems.

2. AI Agent vs. Traditional Chatbot

A traditional chatbot generally receives a user message and generates a response directly from the language model. An AI agent can analyze the request, determine whether additional information is required, select an appropriate tool, execute one or more actions, observe the results, and then formulate a final response.

3. Core Components of an AI Agent

The main components of an AI agent are the model, instructions or prompts, state, tools, memory, and an execution or orchestration layer. The model performs reasoning and language generation. Prompts define behavior. State stores information needed during execution. Tools allow the agent to interact with external systems. Memory allows useful information to persist across interactions. An orchestration framework controls the workflow.

4. Large Language Models

Large language models such as Gemini, GPT, and Claude provide the reasoning and language-generation capability used by many AI agents. An LLM can interpret natural-language instructions, summarize information, classify text, generate structured data, and decide which available tool may be useful. The LLM by itself does not automatically have access to a company's private documents or live external information.

5. Prompts and Instructions

A prompt provides instructions and context to an AI model. A well-designed agent prompt should define the agent's role, objectives, constraints, available tools, expected behavior, and rules for handling missing information. For example, a document assistant can be instructed to answer using retrieved company documents and to say that information is unavailable when the documents do not contain the answer.

6. Tools and Tool Calling

Tools are functions or external capabilities that an AI agent can invoke. Examples include web search, database queries, calculators, APIs, file retrieval, and enterprise services. In tool calling, the model selects a tool and supplies structured arguments. The application executes the tool and returns the result to the model. The model can then use the result to continue the workflow.

7. Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation combines information retrieval with language generation. A user question is converted into an embedding and compared with embeddings stored in a vector database. The most relevant document chunks are retrieved and supplied as context to the language model. The model then generates an answer grounded in that context.

8. RAG Pipeline

A typical RAG pipeline contains document loading, text extraction, cleaning, chunking, embedding generation, vector-store indexing, retrieval, prompt construction, and answer generation. During ingestion, documents are split into smaller chunks and converted into vectors. During a query, the question is embedded, similar chunks are retrieved, and those chunks are passed to the LLM as context.

9. Document Chunking

Chunking divides a large document into smaller pieces that can be retrieved independently.

`RecursiveCharacterTextSplitter` is a common strategy in `LangChain`. Chunk size and overlap affect retrieval quality. Very small chunks may lose important context, while very large chunks may contain unrelated information. Overlap helps preserve information that crosses chunk boundaries.

10. Embeddings

An embedding is a numerical vector representation of text. Texts with similar meanings tend to have vectors that are close together in embedding space. Embeddings allow semantic search, meaning that retrieval can find conceptually related text even when the exact words in the question do not appear in the document.

11. Vector Stores

A vector store indexes embeddings and supports similarity search. `FAISS` is a popular local vector-search library. Other vector databases include `Chroma`, `Pinecone`, `Weaviate`, and `Milvus`. `FAISS` is useful for local development and portfolio projects because it can store and search vectors efficiently without requiring a separate hosted database.

12. Similarity Search

Similarity search retrieves the top-k document chunks that are most similar to a query embedding. With `FAISS`, a distance score can be returned. A lower distance generally indicates greater similarity for the distance metric being used. The score should not be interpreted as a probability or as an LLM confidence percentage.

13. RAG Grounding and Hallucination Control

Grounded RAG attempts to limit hallucination by giving the LLM relevant source context and instructing it to answer from that context. A robust system should avoid inventing facts when the retrieved documents do not contain sufficient information. Retrieval thresholds, source citations, prompt rules, and evaluation tests can improve reliability.

14. LangChain

LangChain is a framework for building applications around language models. It provides abstractions for models, prompts, output parsers, retrievers, vector stores, tools, and runnable pipelines. LangChain Expression Language allows components to be composed with the pipe operator, making multi-step LLM workflows easier to organize.

15. LangGraph

LangGraph is a framework for building stateful, graph-based agent workflows. A LangGraph application represents execution as nodes and edges. Nodes perform operations such as retrieving documents, calling an LLM, or executing a tool. Edges determine how execution moves between nodes. State carries information between steps.

16. State in Agent Workflows

Agent state contains information required during execution. A state can include the user question, conversation messages, retrieved documents, tool results, intermediate decisions, and final answer. TypedDict or Pydantic models can be used to define structured state in Python applications.

17. Messages

Modern LLM applications commonly represent conversations as messages. Typical message types include system messages, human messages, AI messages, and tool messages. Message-based state allows an agent to maintain the sequence of instructions, user requests, model responses, and tool results during an interaction.

18. Agent Memory

Memory allows an application to retain useful information across interactions. Short-term memory keeps information within the current conversation or execution. Long-term memory can persist selected information across sessions. Memory should be designed carefully because storing unnecessary or sensitive information can create privacy and reliability risks.

19. Structured Output

Structured output means asking the LLM to return data that follows a defined schema instead of unrestricted text. Pydantic models, TypedDict, and JSON are common approaches. Structured output is useful for extracting entities, classifying questions, returning tool arguments, and passing reliable data between application components.

20. Sequential and Parallel Chains

A sequential chain executes steps in order, where the output of one step can become the input to the next. A parallel chain executes independent operations concurrently and combines their outputs. For example, an application might generate an explanation and a list of advantages in parallel before combining them into a final response.

21. Agent Decision Making

An agent can route a request based on the task. A document question may require RAG. A question about current events may require web search. A request that needs both private company information and current public information may require multiple tools. Routing should be explicit and observable so developers can understand why a tool was selected.

22. MCP — Model Context Protocol

Model Context Protocol (MCP) is a protocol for connecting AI applications with external tools and data sources through a standardized interface. An MCP server can expose tools or resources, while an MCP client connects an AI application to those capabilities. MCP can reduce the need to build custom integrations for every individual tool.

23. FastAPI in an AI Application

FastAPI can provide an HTTP API layer between a user interface and an AI backend. A typical endpoint accepts a question and optional session information, invokes the agent or RAG pipeline, and returns a structured response. Separating the API from the user interface makes the backend reusable by web applications, mobile applications, and other clients.

24. Streamlit in an AI Application

Streamlit is useful for quickly creating an interactive user interface for AI applications. An InsightAgent interface can provide a chat experience, display answers, show retrieved source documents, expose session history, and provide controls for uploading documents or selecting tools.

25. Evaluation of RAG and Agents

AI applications should be evaluated using representative test questions. RAG evaluation can measure retrieval accuracy, relevance of retrieved chunks, answer correctness, and whether unsupported information was introduced. Agent evaluation can also measure tool-selection accuracy, workflow success, latency, and failure handling.

26. Production Considerations

A production AI application should separate configuration from source code, protect API keys, use environment variables, implement logging, validate inputs, handle errors, monitor latency and token usage, evaluate responses, and control access to private data. Docker can provide a consistent runtime environment, while cloud services can host the API and user interface.

27. Example InsightAgent Workflow

InsightAgent can combine RAG, web search, memory, and MCP tools. For a question about an internal policy, the agent can retrieve relevant company-document chunks and answer from those sources. For a question about current public information, it can use web search. For a question requiring both internal policy and current

information, it can retrieve internal documents and use an external tool before synthesizing the results.

28. Example Questions for Testing Retrieval

The following questions can be used to test InsightAgent: 1. What is an AI agent? 2. How is an AI agent different from a traditional chatbot? 3. What are the core components of an AI agent? 4. What is retrieval-augmented generation? 5. Why is document chunking important? 6. What is an embedding? 7. Why would an application use FAISS? 8. What does a similarity score represent? 9. What is LangGraph? 10. What is the purpose of state in LangGraph? 11. What is tool calling? 12. What is MCP? 13. How can FastAPI be used in an AI application? 14. How can Streamlit be used for an AI assistant? 15. What techniques can reduce hallucinations in a RAG application?